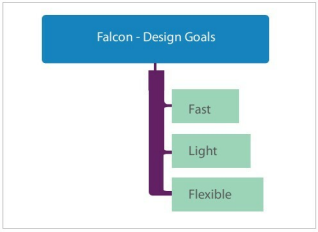

Figure 1: Falcon design goals

Flexible: Falcon doesn't restrict developers when choosing libraries with respect to databases, authorisation, etc. Developers can select their preferred libraries, which match the requirements of the current project's scenario.

Falcon's feature set

Some of the major features of Falcon are listed below:

- Clean and extensible code base
- Simple and quick access to headers and bodies
- Simple and effective exception handling
- Support for various versions of Cpython, PyPy and Jython
- When used with Cython, the average increase in speed has been around 20 per cent

Installation

As stated earlier, Falcon is a high-performance framework. Official documentation recommends the use of PyPy to achieve optimal performance. PyPy is an alternative implementation of Python, which is comparatively faster. Speed, memory usage and compatibility are its major features. To explore further, navigate to *http://pypy.org/*.

The installation of Falcon is simple and quick. It can be installed using Pip, as shown below:

```
$ pip install falcon
```

Web Server Gateway Interface (WSGI)

Falcon communicates through WSGI, which is a specification used for interfacing Web servers and Web applications or frameworks for the Python language. WSGI was built to promote portable Web application development. Two major advantages of WSGI are listed below:

- The use of WSGI increases the flexibility. App developers can swap out Web stack components for others.
- It offers better scaling features. When the number of concurrent requests is great, the WSGI servers facilitate efficient scaling by segregating the responsibilities.

To provide a Falcon app, we need a WSGI server.

Figure 2: WSGI servers

There are many choices listed in *http://wsgi.readthedocs.io/en/latest/servers.html*. The popular ones among the developer community are featured in Figure 2 and listed below.

- *Green Unicorn (Gunicorn):* This is a Python WSGI HTTP server for UNIX. The Gunicorn server is compatible with various Web frameworks. (*http://gunicorn.org/*)
- *uWSGI:* The objective of the uWSGI project is to provide a full stack for building hosting services. (*http://uwsgi-docs.readthedocs.io/en/latest/*)
- *mod_wsgi:* This is an Apache module, which implements the WSGI specification. (*https://github.com/GrahamDumpleton/mod_wsgi*)
- *CherryPy:* This is a Pythonic WSGI server with object-oriented features. (*https://github.com/cherrypy/cherrypy*)
- *Julep:* This is a WSGI server, which has been inspired by Unicorn and is written in pure Python. (*https://code.google.com/archive/p/julep/*)

Installation of any one of these WSGI servers is required for Falcon. You may choose *gunicorn* or *uwsgi*, and install them using the Pip command as shown below:

```
$pip install gunicorn
```

Falcon – a simple example

To run a Falcon app, you simply need to install Falcon and Gunicorn using Pip commands. Let's consider a simple example that displays a message.

```
# things.py
# Import the Falcon
import falcon
class ThingsResource(object):
    def on_get(self, req, resp):
        """Handles GET requests"""
     resp.status = falcon.HTTP_200
     resp.body = ('\nTwo things awe me most, the starry sky '
                  'above me and the moral law within me.\n'
                  '\n'
                  '    ~ Immanuel Kant\n\n')
# falcon.API instances are callable WSGI apps
app = falcon.API()
# Resources are represented by long-lived class instances
things = ThingsResource()
```